



Double laser-faults based PFA on cryptographic circuits with algebraic analysis

Xue Gong^a, Ao Shen^a, Tianxiang Feng^a, Guorui Xu^a, Shize Guo^{a,b}, Fan Zhang^{a,b,*}

^a College of Computer Science and Technology, Zhejiang University, Hangzhou, 310027, China

^b Zhengzhou Xinda Institute of Advanced Technology, Zhengzhou, 450000, China

ARTICLE INFO

Keywords:

Hardware security
Cryptographic circuit
Laser faults
Algebraic analysis
Persistent fault attack

ABSTRACT

Cryptographic algorithms have been employed in a variety of fields as the primary method to protect information security. The security of a cryptographic algorithm is closely related to its operating environment and physical devices. Algebraic Persistent Fault Analysis (APFA) is a new fault analysis method for block ciphers proposed in CHES 2022, which utilizes the fault that persists in encryptions and introduces algebraic analysis in the fault analysis step. In the fault injection step, as the transistors of the integrated circuit are getting smaller and tighter, even high-precision devices may cause more than one fault per injection. However, more faults may lead to a more efficient attack in the fault analysis step. In this paper, APFA for double faults is proposed, which can deal with the double faults model and reduce the number of required ciphertexts. The practicality of our fault injection is validated by laser fault injection experiments on the SRAM embedded in an ATmega163L microcontroller. The effectiveness of our fault analysis is proven by successfully recovering the key of PRESENT-128 and AES-128. The number of ciphertexts needed for key recovery is reduced by 46% compared to PFA with a single fault.

1. Introduction

With the continuous development of integrated circuits and electronic information technology, the industry and society have become more and more intelligent. The information security plays a more decisive role. As a result, the importance of information security has increased. As the main tool to protect information security, cryptographic systems have been used in various fields including economy, finance, politics, etc. In practical applications, cryptographic algorithms are mapped to hardware or software. So, the security of cryptographic algorithms is closely related to the operating environment and physical devices.

Side-channel attacks (SCA) and Fault attacks (FA) are two classes of attacks on cryptographic circuits. Unlike traditional cryptographic analysis, SCA and FA utilize the near-field device to gather extra information which contains the key information. In SCA, an attacker deduces the secret key by observing information that leaks during the normal functioning of the device [1,2]. In FA, the attacker actively injects faults into the device and recovers the secret key by examining the response of the device to the faults [3–5]. In this paper, we focus on fault attack, which consists of two steps: fault injection (FI) and fault analysis.

1.1. Fault injection

In the fault injection step, depending on the duration, faults can be classified as transient, permanent, or persistent. Transient faults can be injected by creating a voltage drop in the circuit [6] or by Trojans, clock glitches [7], laser pulses [8], etc. To inject a transient fault, the attacker needs to get precise control of the fault intensity and time precision, which makes it hard to achieve in practical situations. Permanent faults are caused by intentional or unintentional defects in chips [9]. Since the permanent defects are typically irrecoverable, attackers rarely employ them in practice.

The original idea of persistent fault attack or analysis (PFA) was given by Zhang et al. in CHES 2018 for cracking block ciphers [10]. In their fault model, the fault persists from one encryption to another until the target device reboots. In the fault injection step, precise timing is not required, which means the attacker does not need to maintain synchronization with the encryption algorithm.

Fig. 1 is the schematic of persistent fault injection by the laser pulse. The pulsed laser beam is emitted to the targeted device, for example, a Microcontroller Unit, by the microscope system. For block ciphers, by shooting laser pulses into the targeted cryptographic circuits, the adversary can inject faults into the S-Box.

* Corresponding author at: College of Computer Science and Technology, Zhejiang University, Hangzhou, 310027, China.

E-mail address: fanzhang@zju.edu.cn (F. Zhang).

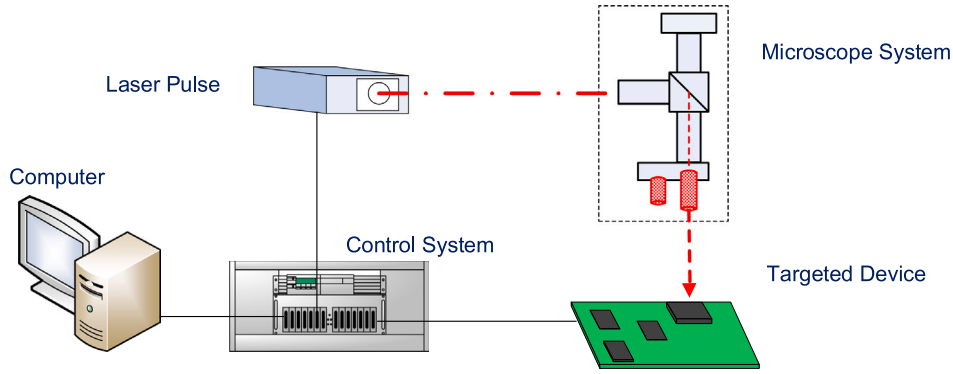


Fig. 1. Schematic of persistent fault injection.

1.2. Fault analysis

In the fault analysis step, the adversary examines the response of the device to the injected faults and attempts to recover the secret key. Differential analysis, statistical analysis, and algebraic analysis are among the most effective analysis methods in this step.

Traditional differential analysis is a type of chosen plaintext attack [11]. The attacker needs to obtain a pair of plaintexts P_1, P_2 , and its corresponding pair of ciphertexts C_1, C_2 . By examining the influence of plaintext difference ($\Delta P = P_1 \oplus P_2$) to ciphertext difference ($\Delta C = C_1 \oplus C_2$), the attacker tries to derive the key. Biham and Shamir adopted the idea of differential analysis and proposed Differential Fault Analysis (DFA) on block cipher DES [12]. In the fault attack scenario, DFA gets a correct ciphertext C_{correct} and a faulty ciphertext C_{faulty} corresponding to the same plaintext and uses the difference of them ($\Delta C = C_{\text{correct}} \oplus C_{\text{faulty}}$) to recover the key. It can be applied to AES [13–15], PRESENT [16] and other block ciphers [17].

Statistical analysis focuses on the biased distribution of ciphertexts. In [10], Zhang et al. performed statistical analysis on the distribution of the faulty ciphertexts to recover the key. In [18], a method based on maximum likelihood estimation (MLE) was utilized to reduce the required ciphertexts. Other works that exploited statistical fault analysis include Statistical Fault Analysis (SFA) [19] and Statistical Ineffective Fault Analysis (SIFA) [20], etc.

When a persistent fault is injected into the device, it persists in every round of the block cipher. Exploiting the information leakage in deeper rounds can help to reduce the ciphertexts we need in the fault analysis step. However, differential analysis and statistical analysis are ineffective in exploiting information leaks in deeper rounds. To improve the efficiency of fault analysis, algebraic cryptanalysis is a good choice.

In algebraic cryptanalysis, the main idea is to convert the cryptographic algorithm to algebraic equations and recover the key by solving these equations. In eSmart 2010, Courtois et al. [21] combined algebraic analysis with fault attack and proposed a new fault analysis technique called algebraic fault analysis (AFA). In AFA, the fault information is represented as algebraic equations and these equations are added to the equation system of the encryption algorithm. In CHES 2022, Zhang et al. proposed Algebraic Persistent Fault Analysis (APFA) [22], which introduces algebraic techniques into the fault analysis step of PFA. It can exploit multi-round fault leakage and reduce the number of required ciphertexts.

1.3. Related works

Apart from the previously described works, for multiple faults scenario, Engels et al. presented Statistical Persistent Fault Analysis (SPFA) by merging Statistical Fault Analysis (SFA) with PFA [23]. SPFA can bypass the premise that multiple fault values and locations are known and can reduce the search space of keys under multiple faults. However, SPFA can only reduce the search space of the key to 2^{50} . Soleimany

et al. proposed a practical analysis method named Practical Multiple Persistent Faults Analysis (PMPFA) to overcome the problems of the high computational complexity of SPFA [24]. With double faults injected, it can reduce the key candidates to 2^{23} with 1552 ciphertexts. While in our work, we only need 700 ciphertexts to reduce the key candidate to one.

1.4. Motivation and challenges

In the fault injection step, as the process nodes of the integrated circuit are getting smaller, the fault injection process requires higher accuracy. In practice, even high-precision fault injection methods like laser pulse may induce more than one fault per injection. We observe the case from the real laser-inducing experiment on the SRAM embedded on an ATmega163L microcontroller [18]: two faults in the S-box can be caused simultaneously by one laser injection.

In the fault analysis step, adding a fault means leaking more information about the cryptographic circuits. By utilizing it efficiently, we can recover the key more easily. So, if we want to further relax the requirements for accuracy in the fault injection step and reduce the number of ciphertexts in the fault analysis step, APFA under a single fault should be extended. And it is natural to think of how to use APFA under double faults. In [10,24], multiple faults models are discussed, but the analysis method they use is a statistical analysis method which is different from algebraic analysis. Algebraic analysis has been proven a very effective analysis method, but how to use algebraic analysis under the double faults model has never been explored.

For SPN block cipher, the premise of using algebraic analysis is knowing the truth table of the S-box. However, in the process of fault injection, the injector cannot accurately control the fault locations and the fault values. Therefore, theoretical analysis should be made for all the scenarios of whether the fault locations and fault values are known or not. For the double faults model, this theoretical analysis is more challenging and groundbreaking.

1.5. Our contributions

Motivated by the two challenges above, in this paper, we exploit algebraic analysis for the double faults model of PFA, and the main contributions can be summarized as follows:

1. We propose the Double Faults PFA using algebraic techniques.
2. As the algebraic representation of the S-box is essential in APFA on SPN block ciphers, in this paper, for the representation of the S-box, four different scenarios are elaborated according to whether the fault values and the fault locations are known or not.

3. We simulate these four scenarios and use APFA to recover the key successfully. We experimented with two mainstream ciphers, PRESENT-128 and AES-128, among which PRESENT-128 reduces the number of ciphertexts by 46.43% compared to a single fault. And AES-128 only needs 700 ciphertexts to recover the full key.

1.6. Structure of this paper

Section 2 presents preliminaries, including SPN block cipher, the notations in this paper, PFA, algebraic cryptanalysis, and APFA. In Section 3, we build the double faults model and discuss the four scenarios of whether the fault locations and fault values are known or not. For each scenario, we give the number of enumerations. Section 4 is the detail of algebraic analysis for double faults which can be applied to all SPN block ciphers. Section 5 presents our experimental results.

2. Preliminaries

In this section, we introduce the background knowledge of SPN block cipher with the notations we use in the following sections. We give a short description of Persistent Fault Attacks, algebraic cryptanalysis, and Algebraic Persistent Fault Analysis.

2.1. SPN block cipher and notations

Substitution Permutation Network (SPN) is a commonly used structure in block ciphers. Henceforth, we will use this structure to illustrate our fault model so we fix some notations for convenience.

We denote P for plaintext and C for ciphertext. The SPN block cipher is denoted as $\mathbb{B}$. Its block size is n bits, and each element in the block is of w bits. For AES, each element in the S-box is of 8 bits, so we have $w = 8$ (i.e., a byte) and for PRESENT we have $w = 4$ (i.e., a nibble). The block cipher has R rounds. X^r, Y^r are the input and output data blocks for a certain function in the r th round. K^r is the r th round key. X_j^r, Y_j^r, K_j^r are the j th element of X^r, Y^r and K^r respectively. For example, in AES, X^r has 128 bits and each X_j^r has 8 bits. x_i^r, y_i^r and k_i^r are the i th bit of X_j^r, Y_j^r and K_j^r . If not for a specific round, we omit the superscript r and denote like $X, Y, K, X_j, Y_j, K_j, x_i, y_i, k_i$ for simplicity.

The round function F consists of three parts:

1. Substitution (SB). The input data block is X and the output data block is Y . $Y = SB(X)$ means the substitution of X according to the S-box S . $S[l]$ is the l th element of the S-box. To ensure correct decryption, the function must be invertible. We denote the inversion of it as SB^{-1} .
2. Permutation Layer (PL). If the permutation is bit-based, like in the block cipher PRESENT, the operation PL can be regarded as an index mapping: $y_j = x_i$, y_j is the j th bit of the output Y , x_i is the i th bit of the input X . The inverse operation of PL is PL^{-1} . We denote $PL^{-1}(Y^r)$ as $\tilde{Y}^r$, $PL^{-1}(K^r)$ as $\tilde{K}^r$ and $PL^{-1}(C)$ as $\tilde{C}$.
3. Add Round Key (AK). This operation applies the round key to the data block with simple modulo 2 additions: $Y = X \oplus K$. The round key is generated by the key expansion function.

Table 1 shows the notations used in this paper.

2.2. Persistent fault attack

In Persistent Fault Attack, the attacker aims at the substitution table. In the fault injection step, the attacker injects a fault into the S-box, for example, by exposing the device to a laser. In the fault analysis step, the attacker uses the collected faulty ciphertexts to recover the key, for example, in [10], they employed the statistical analysis method in this step.

The persistent fault injection into the S-box is depicted in Fig. 2. The upper part of Fig. 2 is the physical location of an S-box element in

Table 1

Notations used in this paper.

Notations	Definitions
P, C	The plaintext and ciphertext
$\mathbb{B}$	The block cipher
n	The block size
w	The element size
r	The r th round
R	The total number of rounds
X^r, Y^r, K^r	The r th round data block and the r th round key
X_j^r, Y_j^r, K_j^r	The j th element of X^r, Y^r, K^r , $0 \leq j < \frac{n}{w}$
x_i^r, y_i^r, k_i^r	The i th bit of X^r, Y^r, K^r , $0 \leq i < w$
F	The round function
SB, SB^{-1}, SB'	Substitution, Inversion of Substitution, Faulty Substitution
S, S'	S-box, faulty S-box
f	The fault value
$S[l]$	The l th element of S-box, l is the fault location
V_l	The value of $S[l]$
PL, PL^{-1}	Permutation layer and Inverse permutation layer
$\tilde{Y}^r, \tilde{K}^r, \tilde{C}$	$PL^{-1}(Y^r), PL^{-1}(K^r), PL^{-1}(C)$
$C_j, \tilde{C}_j$	The j th element of C and $\tilde{C}$
$\tilde{C}_j^{max}, \tilde{C}_j^{min}$	The value of $\tilde{C}_j$ with the maximum and minimum frequency

SRAM [18]. The 6th bit of the element is flipped from 1 to 0 due to the laser pulse. As a result, the value of the element is changed from $0 \times 7c$ to $0 \times 5c$.

For the general situation, let us assume a persistent fault is injected into the S-Box S , and the l th element of the S-box $S[l]$ is affected. The correct value $S[l] = V_l$ will be changed to $S[l] = V_l \oplus f$ with a fault value f , $f \neq 0$. It can be found that $V_l \oplus f$ appears with doubled frequency in the output of S-box, and V_l no longer appears. To simplify our illustration, in this section, we focus on the last round of encryption as in the original PFA. SB' is the Substitution with faulty S-box. As shown in Fig. 3, for SPN block ciphers, we have $C = PL(SB'(X^R)) \oplus K^R$, which is equivalent to $C \oplus K^R = PL(SB'(X^R))$. By applying the PL^{-1} function and according to the property of PL^{-1} , we have $PL^{-1}(C) = PL^{-1}(K^R) \oplus SB(X^R)$, i.e.,

$$\tilde{C} = \tilde{K}^R \oplus SB'(X^R) \quad (1)$$

For the j th element in Eq. (1):

$$\tilde{C}_j = S'[X_j] \oplus \tilde{K}_j^R \quad (2)$$

Let $\tilde{C}_j^{min}$ denote the value that should never appear in the specific $\tilde{C}_j$, then,

$$\tilde{C}_j^{min} = V_l \oplus \tilde{K}_j^R \quad (3)$$

Let $\tilde{C}_j^{max}$ denote the value of $\tilde{C}_j$ that should appear with doubled frequency where

$$\tilde{C}_j^{max} = V_l \oplus f \oplus \tilde{K}_j^R \quad (4)$$

If sufficient ciphertexts are collected, by applying PL^{-1} to them, we can find $\tilde{C}_j^{min}$ and $\tilde{C}_j^{max}$. As we have $K^R = PL(\tilde{K}^R)$, by recovering $\tilde{K}^R$, we can get K^R . Three possible strategies can be applied to recover $\tilde{K}_j^R$.

Strategy 1: Utilizing $\tilde{C}_j^{min}$. If we know V_l , $\tilde{K}_j^R$ can be directly deduced as in Eq. (5).

$$\tilde{K}_j^R = \tilde{C}_j^{min} \oplus V_l \quad (5)$$

Strategy 2: Utilizing $\tilde{C}_j^{max}$. If we know $V_l \oplus f$, $\tilde{K}_j^R$ can be easily computed as in Eq. (6).

$$\tilde{K}_j^R = \tilde{C}_j^{max} \oplus V_l \oplus f \quad (6)$$

Strategy 3: Utilizing $\tilde{C}_j \neq V_l \oplus \tilde{K}_j^R$. For all collected ciphertext $\tilde{C}_j$, the search space of $\tilde{K}_j^R$ can be reduced by Eq. (7).

$$\tilde{K}_j^R \neq V_l \oplus \tilde{C}_j \quad (7)$$

In practice, **Strategy 3** is the most commonly adopted.

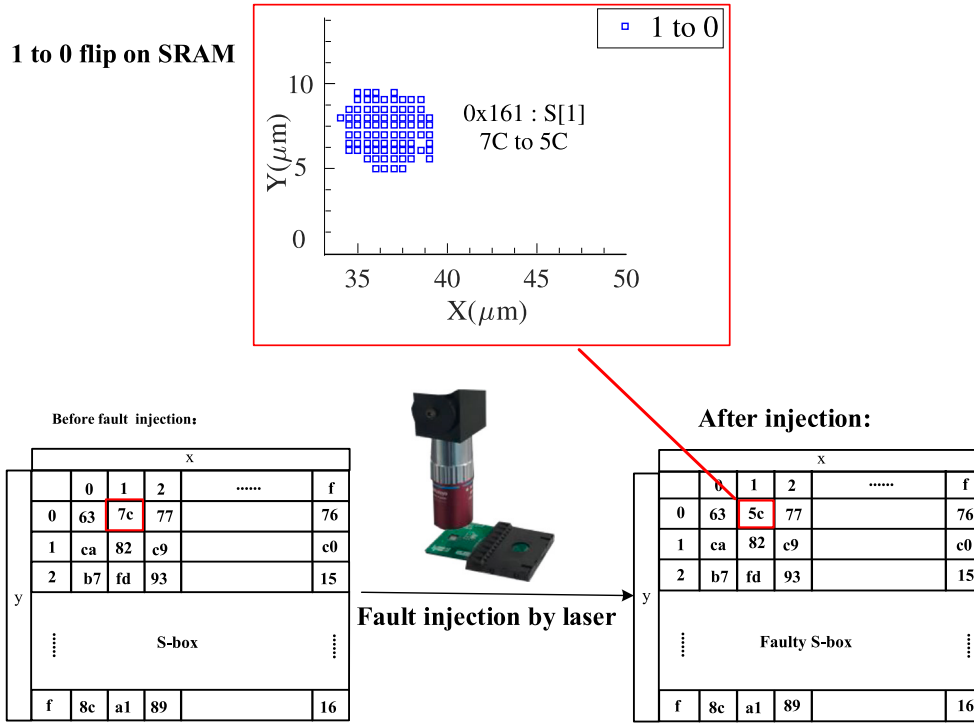


Fig. 2. Persistent fault injection into the S-box.

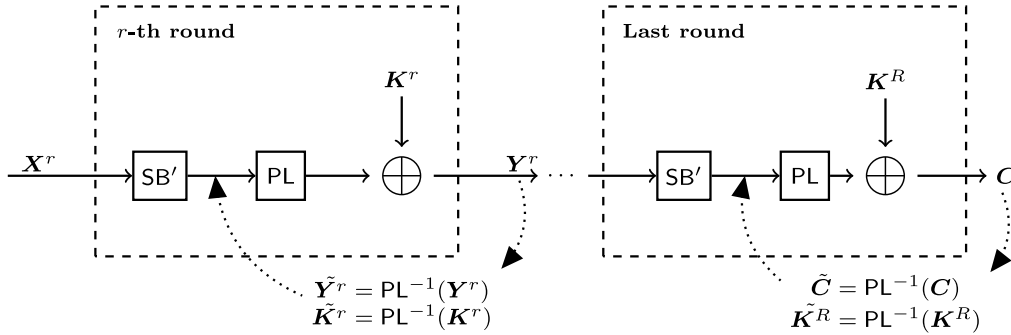


Fig. 3. An illustrative SPN block cipher.

2.3. Algebraic cryptanalysis

As shown above, the depiction of the relationship between input and output is not highly sophisticated in the last round. What about multiple rounds? In fact, it becomes a challenge to analyze the relationship as the round goes deeper. Algebraic cryptanalysis provides a solution to this challenge.

Algebraic cryptanalysis is mainly composed of two steps: the first step is to build the algebraic equations of the cryptographic algorithm; the second step is to solve the equations and use the solutions to recover the key.

However, as the cryptographic algorithms are designed to prevent us from getting plaintext information or the keys from ciphertexts, it is not easy to solve these equations in an affordable time. So, in algebraic analysis, the most important thing is to reduce the computational complexity of solving equations.

There are three common methods in algebraic analysis. They are based on different mathematical problems. The first one is the method of Gröbner basis, which can effectively reduce the order of multivariate polynomials [25]. The second one is based on the linearization principle (XL) [26]. The third one is to transform the problem into the satisfiability problem (SAT) [27]. That is, to tell whether there is a

set of Boolean variables that meet specific requirements. According to the characteristics of the equations, each of the three methods has its advantages. In this paper, we use the third method, solving the SAT problem.

In recent years, researchers who focus on solving algebraic equation systems have developed solvers for satisfiability problems (Minisat [28], CryptoMinisat [27], etc.). To some extent, these solvers can be regarded as a search tool for solutions.

2.4. Algebraic persistent fault analysis

Algebraic Persistent Fault Analysis (APFA) [22] combines the algebraic cryptanalysis with PFA. The core idea of APFA is to use the algebraic system to represent **Strategy 3** and it can use the information leakage in multiple rounds. Firstly, APFA establishes algebraic equations from the final round back to the r th round. Secondly, and most importantly, APFA establishes the equations of fault leakages for each round. Finally, APFA uses an algebraic solver to get the key. For the second step, as the notations in Table 1, Y_j^r is the j th element of the block after the key addition in the r th round and K_j^r is the j th element of the r th round key. The constraint used to reduce the round key search

space of PFA is just the same as in **Strategy 3** but it can be generalized by Eq. (8).

$$\tilde{K}_j^r \neq V_l \oplus \tilde{Y}_j^r \quad (8)$$

The goal of APFA is to use Eq. (8) under algebraic analysis. If $A \neq B \oplus C$, it means $A \oplus B \oplus C \neq 0$. Through this property, Eq. (8) can be represented algebraically, as shown in Eq. (9), where $\tilde{y}_i$, $\tilde{k}_i$ and v_i are the i th bit of $\tilde{Y}_j^r$, $\tilde{K}_j^r$ and V_l , respectively, and w is the number of bits in $\tilde{Y}_j^r$, $\tilde{K}_j^r$ and V_l .

$$\tilde{y}_i \oplus \tilde{k}_i \oplus v_i \neq 0, \quad 0 \leq i < w \quad (9)$$

In short, for each ciphertext, we can build a set of equations, and assign values to the variables in the equation system. When building equations for each ciphertext, the corresponding constraint as in Eq. (9) are added for the last $n - r + 1$ rounds at the same time. Compared with PFA, it is able to significantly reduce the number of ciphertexts required, and exploit as many rounds of fault leakages as possible.

3. Our analysis framework

In this section, we introduce the double faults model. To prepare for algebraic analysis, we discuss how to get the faulty S-box. According to whether the fault locations and fault values are known or not, there are totally four different scenarios. For each scenario, we give the number of enumeration.

3.1. Fault model

The double faults model is as follows:

- The adversary injects two faults before the encryption of a block cipher. These faults affect two stored algorithm constants, for example, changing two elements in S-box.
- The injected faults persist in multiple rounds of encryption. The victim uses a fixed key to encrypt multiple plaintexts via the faulty device.
- The adversary collects the faulty ciphertext and aims to recover the key.

We assume the two faults are injected into the l_1 th and l_2 th element of S, and the fault values are f_1 and f_2 respectively, $f_1, f_2 \neq 0$. These four values must satisfy the relationship as Eq. (10).

$$\begin{cases} S[l_1] \oplus f_1 \neq S[l_2] \\ S[l_2] \oplus f_2 \neq S[l_1] \end{cases} \quad (10)$$

If not so, as in Eq. (11), the faults just change the position of two values in the S-box. This will not influence the distribution of ciphertexts. So it is not within the scope of our analysis.

$$\begin{cases} S[l_1] \oplus f_1 = S[l_2] \\ S[l_2] \oplus f_2 = S[l_1] \end{cases} \quad (11)$$

Also, as in Eqs. (12) and (13), it is just the same case as a single fault, which has been discussed in [22].

$$\begin{cases} S[l_1] \oplus f_1 \neq S[l_2] \\ S[l_2] \oplus f_2 = S[l_1] \end{cases} \quad (12)$$

$$\begin{cases} S[l_1] \oplus f_1 = S[l_2] \\ S[l_2] \oplus f_2 \neq S[l_1] \end{cases} \quad (13)$$

The feasibility of our fault model is verified by the laser fault injection experiments in [18]. The experiment is conducted on the software implementation of AES running on an ATmega163L microcontroller. The data of S-Box is loaded during the boot-up before the fault injection. The adversary can inject two faults into the SRAM region of the microcontroller with one laser pulse.

Just as in Fig. 4, which is an example of double faults injection into the S-box of a block cipher. After the fault injection by the adversary, two elements in the S-box are changed, the first one altered from 0×79 to $0 \times e9$, the second one altered from $0 \times 7d$ to $0 \times 5d$. The upper part of Fig. 4 is the result in [18]. This indicates that it is possible to inject two faults simultaneously with only one laser pulse. One of the possible regions to position the laser beam is shown as the black box, which will cause both S[18] and S[19] to change at the same time.

3.2. Double faults exploitation

APFA adds constraints to the round keys by using Eqs. (8) and (9). It can be found that in Eqs. (8) and (9), V_l is a constant independent of other intermediate variables. Suppose V_1 and V_2 are the values of $S[l_1]$, $S[l_2]$ respectively. Since neither V_1 nor V_2 will appear in the output of the S-box, they will both satisfy $\tilde{K}_j \neq \tilde{Y}_j \oplus V_1$, $\tilde{K}_j \neq \tilde{Y}_j \oplus V_2$.

3.3. Identify fault locations and fault values

In APFA, it is necessary to know the truth table of the faulty S-box S' because we need to use the algebraic representation of it. Therefore, the key point for APFA with double faults is to identify fault locations l_1, l_2 and fault values f_1, f_2 . We will discuss the four cases of whether they are known or not.

3.3.1. Case one

l_1, l_2 are known and f_1, f_2 are known. This case is not different from APFA under a single fault. The values of V_1, V_2 can be obtained by l_1, l_2 .

3.3.2. Case two

l_1, l_2 are known and f_1, f_2 are unknown. Let us assume $l_1 = 1, l_2 = 2$, $S[1] = V_1, S[2] = V_2$. By collecting the ciphertexts C and doing the PL^{-1} to them, we get $\tilde{C} = PL^{-1}(C)$. For each element $\tilde{C}$ under our double faults model, the probability distribution is shown in Eq. (14).

$$Pr(\tilde{C}_j) = \begin{cases} 0 & \tilde{C}_j = V_1 \oplus \tilde{K}_j \\ 0 & \tilde{C}_j = V_2 \oplus \tilde{K}_j \\ 2 \times 2^{-w} & \tilde{C}_j = V_1 \oplus f_1 \oplus \tilde{K}_j \\ 2 \times 2^{-w} & \tilde{C}_j = V_2 \oplus f_2 \oplus \tilde{K}_j \\ 2^{-w} & \text{otherwise} \end{cases} \quad (14)$$

By analyzing the values of $\tilde{C}_j$, we get two values with the maximum frequency and we denote them as $\tilde{C}_j^{max_1}$ and $\tilde{C}_j^{max_2}$. Also there are two values that should never appear, and we denote them as $\tilde{C}_j^{min_1}$ and $\tilde{C}_j^{min_2}$. We need to get the values of f_1 and f_2 from these four values.

For the single fault model, we get

$$\tilde{C}_j^{max} = V \oplus f \oplus \tilde{K}_j \quad (15)$$

$$\tilde{C}_j^{min} = V \oplus \tilde{K}_j \quad (16)$$

By Eq. (15) $\oplus$ Eq. (16), we have

$$\tilde{C}_j^{max} \oplus \tilde{C}_j^{min} = f \quad (17)$$

Now we need to generalize Eq. (17) to double faults. For double faults model,

$$\begin{aligned} f &= \tilde{C}_j^{max} \oplus \tilde{C}_j^{min} \\ \tilde{C}_j^{max} &\in \{\tilde{C}_j^{max_1}, \tilde{C}_j^{max_2}\} \\ \tilde{C}_j^{min} &\in \{\tilde{C}_j^{min_1}, \tilde{C}_j^{min_2}\} \end{aligned} \quad (18)$$

So there are $2 * 2 = 4$ possible values for f :

$$f \in \{\tilde{C}_j^{max_1} \oplus \tilde{C}_j^{min_1}, \tilde{C}_j^{max_1} \oplus \tilde{C}_j^{min_2}, \tilde{C}_j^{max_2} \oplus \tilde{C}_j^{min_1}, \tilde{C}_j^{max_2} \oplus \tilde{C}_j^{min_2}\} \quad (19)$$

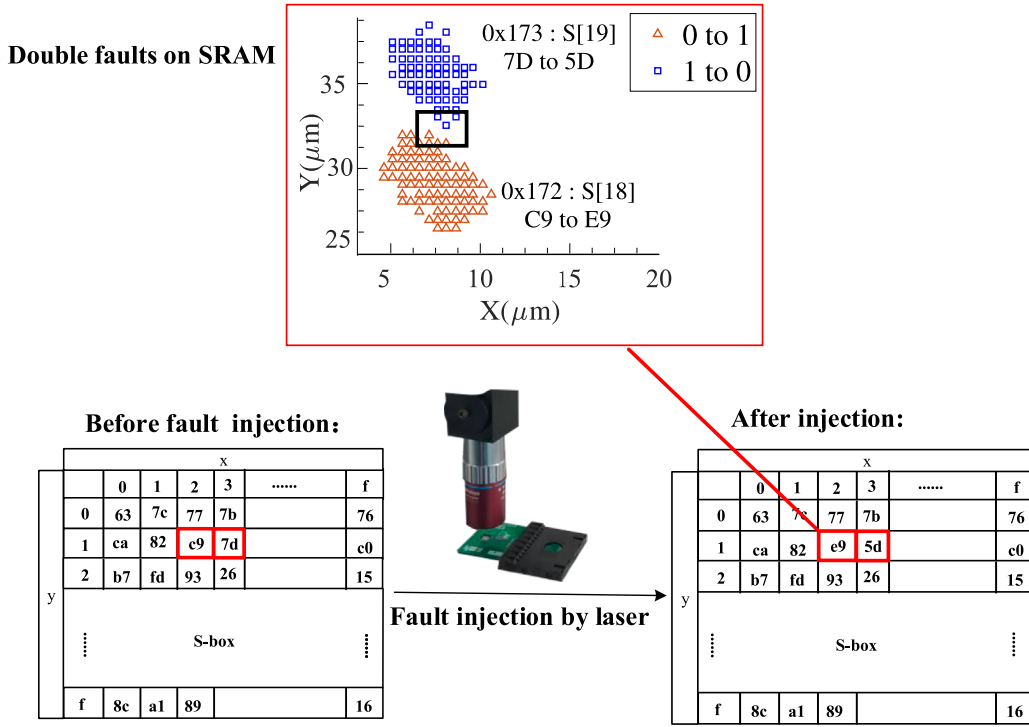


Fig. 4. Double faults injection into the S-box.

Once the value of f_1 is assumed, the value of f_2 is fixed. For example, if we assume $f_1 = \tilde{C}_j^{max_2} \oplus \tilde{C}_j^{min_1}$ then the values left $\tilde{C}_j^{max_1}$ and $\tilde{C}_j^{min_2}$ must be caused by fault f_2 , which means $f_2 = \tilde{C}_j^{max_1} \oplus \tilde{C}_j^{min_2}$. So, there are four cases for the set $\{f_1, f_2\}$.

However, there is a prerequisite that the number of ciphertexts to be analyzed needs to be sufficient. When pursuing the least number of ciphertexts that are required, the technique of *Maximum Likelihood Estimation* (MLE) in [18] can be adopted. In [18], MLE is used to estimate the fault value f from the ciphertexts. MLE for f can be calculated as in Eq. (20) which is detailed in [18].

$$\begin{aligned} \hat{f} &= \arg \max_{\theta} Pr(C_0 C_1 \dots C_{m-1} | f = \theta) \\ &= \arg \max_{\theta} \prod_{i=0}^{m-1} \left(\sum_{j=0}^{2^w-1} Pr(C_j | C_j^{min} = j \wedge C_j^{max} = j \oplus \theta) \right) \end{aligned} \quad (20)$$

It can be seen from Eq. (20) that the estimated value of f is directly related to C_j^{min} and C_j^{max} . Eq. (20) is based on the MLE under a single fault, and if the formula is still used under double faults, the result will lead to a higher probability of the four cases in Eq. (19) than the other possibilities. Therefore, by MLE, we can obtain the value of Eq. (19) without increasing the number of ciphertexts, and continue the analysis as described above.

3.3.3. Case three

l_1, l_2 are unknown and f_1, f_2 are known. We denote $l_1 = p, l_2 = q, S[l_1] = V_p$ and $S[l_2] = V_q$. If we can collect sufficient ciphertexts, we can find the four values $\{\tilde{C}_j^{max_1}, \tilde{C}_j^{max_2}\}$ and $\{\tilde{C}_j^{min_1}, \tilde{C}_j^{min_2}\}$. We get

$$\tilde{C}_j^{min_1} \oplus \tilde{C}_j^{min_2} = V_p \oplus \tilde{K}_j \oplus V_q \oplus \tilde{K}_j = V_p \oplus V_q \quad (21)$$

Which means

$$V_q = \tilde{C}_j^{min_1} \oplus \tilde{C}_j^{min_2} \oplus V_p. \quad (22)$$

For $S[l_1] = V_p$, there are 2^w possible candidate. Once V_p is chosen, V_q is determined by Eq. (22). Because the original S-box is known, we can get the value of l_1 and l_2 from V_p and V_q . So, to get the faulty S-box, we need to enumerate 2^w cases for all the V_p and V_q . For each case, the

fault value can be f_1 or f_2 . So, the total cases we need to enumerate is 2×2^w .

3.3.4. Case four

l_1, l_2 are unknown and f_1, f_2 are unknown. The four values in Eq. (19) can also be recovered by the collection of sufficient ciphertexts or by MLE. Since l_1, l_2, f_1, f_2 are all unknown, according to case2 and case3, there are $2^w \times 4$ possible cases in total.

However, if we loosen the ciphertext-only requirement, more specifically, we allow the attacker to choose the plaintexts, which can greatly reduce the search space of the faults. Firstly, we prepare a set of 2^w plaintexts $\mathbb{P} = \{P^0, P^1, \dots, P^{2^w-1}\}$. The first element (for AES we mean the first byte and for PRESENT we mean the first nibble) of the plaintext ranges from 0 to $2^w - 1$, and the other remaining elements are fixed the same. Then we encrypt these 2^w plaintexts and observe the ciphertexts. We will find two pairs of ciphertexts which are the same:

$$\begin{cases} C^a = C^b \\ C^d = C^e \end{cases} \quad (23)$$

This is due to the double faults injection. We denote $P^a \oplus K^0 = \tilde{P}^a, K^0$ is the first round key added to the plaintext before substitution, and the first element of $\tilde{P}^a$ is $\tilde{P}_0^a$. The corresponding plaintexts satisfy the equation below:

$$\begin{cases} \tilde{P}_0^a = l_1 \\ \tilde{P}_0^b = l_p \end{cases} \quad \begin{cases} \tilde{P}_0^d = l_2 \\ \tilde{P}_0^e = l_q \end{cases} \quad (24)$$

Let $\Delta_1 = P_0^a \oplus P_0^b = l_1 \oplus l_p$, and similarly, $\Delta_2 = P_0^d \oplus P_0^e = l_2 \oplus l_q$. Combining Δ_1, Δ_2 and the four values obtained from MLE, Algorithm 1 can be used to reduce the search space for fault locations and fault values.

4. Algebraic analysis for our double faults model

In an algebraic attack, it is a critical step to represent the cryptographic algorithm as a system of equations. In our double faults model,

Algorithm 1 Pseudo code to distinguish f_1, f_2, l_1, l_2

Input: $value(array), \Delta_1, \Delta_2$, where $value(array)$ is the four values obtained from MLE;

Output: $condi$, the possible value of $\{f_1, f_2, l_1, l_2\}$;

```

repeat
  for  $l_1 = 0$  to  $2^w - 1$  do
    for  $l_2 = l_1 + 1$  to  $2^w - 1$  do
       $f_1 \leftarrow value[0]$ ;
       $f_2 \leftarrow value[1]$ ;
       $V_{xor} \leftarrow f_1 \oplus value[2]$ ;
      if  $S[l_1] \oplus S[l_2] = V_{xor}$  then
        if  $S[l_1] \oplus S[l_2 \oplus \Delta_1] = f_1$  and  $S[l_2] \oplus S[l_2 \oplus \Delta_2] = f_2$  then
           $condi \leftarrow \{f_1, f_2, l_1, l_2\}$ ;
        end if
      end if
    end for
  end for
until  $value \leftarrow next\_permutation(vaule)$ ;
return  $condi$ 

```

besides the representation of the cryptographic algorithm, we also need to consider how to add fault information to the equation system. In this section, we will give the details of algebraic analysis for double faults. We start with the algebraic representation of the SPN block cipher, then we introduce the representation for double faults. Our method can be applied to all SPN block ciphers. To make the description simpler and more specific, we use PRESENT as an example.

4.1. SAT solvers

SAT solvers like CryptoMiniSAT, support *Conjunctive Normal Form* (CNF) and XOR operations. The conjunctive normal form refers to the assumption that there are two variables, a and b , whose values are true or false, and they satisfy the Boolean equation, for example:

$$(a \vee \neg b) \wedge \neg a = true \quad (25)$$

In this case, to make the formula true, we have $a = \text{false}$ and $b = \text{false}$. The conjunctive normal form is composed of several disjunctive clauses, in Eq. (25), $(a \vee \neg b)$ is a disjunctive clause.

To use SAT solvers in our algebraic attack, we need to convert the equations into a set of CNF or XOR clauses.

4.2. Algebraic representation of SPN block cipher

For SPN block ciphers, AddRoundKey, Substitution, and Permutation are the three major operations.

4.2.1. Representation of add round key

AddRoundKey is the bitwise exclusive or (XOR) of the input block X and the key K . The output is Y . x_i, y_i and k_i are one bit of X, Y and K , respectively. The XOR clauses as in Eq. (26) can be added to the solver to represent AddRoundKey.

$$x_i + k_i + y_i = 0, \quad 0 \leq i < n \quad (26)$$

4.2.2. Representation of substitution

The substitution layer is non-linear. It maps the value of X_i to Y_i using the S-box as a truth table. It cannot be added to the solver directly. As in [29], the S-box can be transformed into CNF in two steps. Firstly, we represent the S-box with *Algebraic Normal Form* (ANF). Then, we transform ANF to CNF.

Algebraic Normal Form. X_i and Y_i are the input and output of the S-box. We denote the bitwise representation as $X_i = \{x_0 \| x_1 \| \dots \| x_{w-1}\}$

Table 2

Intermediate variables for one element in PRESENT S-box.

0	x_0	x_1	x_2
I_0	I_1	I_2	I_3
x_3	$x_0 x_1$	$x_0 x_2$	$x_0 x_3$
I_4	I_5	I_6	I_7
$x_1 x_2$	$x_1 x_3$	$x_2 x_3$	$x_0 x_1 x_2$
I_8	I_9	I_{10}	I_{11}
$x_0 x_1 x_3$	$x_0 x_2 x_3$	$x_1 x_2 x_3$	$x_0 x_1 x_2 x_3$
I_{12}	I_{13}	I_{14}	I_{15}

and $Y_i = \{y_0 \| y_1 \| \dots \| y_{w-1}\}$. Then every bit of Y_i can be represented by the bits in X_i in ANF as shown in Eq. (27).

$$y_i = \sum_{p=0}^{2^w-1} a_p x_0^{t_p^0} x_1^{t_p^1} \dots x_{w-1}^{t_p^{w-1}} \quad (27)$$

where $p = \{t_0^p \| t_1^p \| \dots \| t_{w-1}^p\}$ is the binary expansion of p . a_p is the coefficient where $a_p \in \{0, 1\}$. Also $t_i^p \in \{0, 1\}$, where $x_i^{t_i^p}$ ($t_i^p = 1$) means the bit x_i does appear in the monomial $a_p x_0^{t_p^0} x_1^{t_p^1} \dots x_{w-1}^{t_p^{w-1}}$, and $x_i^{t_i^p}$ ($t_i^p = 0$) means x_i does not appear.

For block cipher PRESENT, each element of the S-box is of 4 bits, $w = 4$. The ANF of it is in Eq. (28).

$$\begin{aligned}
 y_0 &= x_0 + x_2 + x_3 + x_1 x_2 \\
 y_1 &= x_1 + x_3 + x_1 x_3 + x_2 x_3 + x_0 x_1 x_2 + x_0 x_1 x_3 + x_0 x_2 x_3 \\
 y_2 &= 1 + x_2 + x_3 + x_0 x_1 + x_0 x_3 + x_1 x_3 + x_0 x_1 x_3 + x_0 x_2 x_3 \\
 y_3 &= 1 + x_0 + x_1 + x_3 + x_1 x_2 + x_0 x_1 x_2 + x_0 x_1 x_3 + x_0 x_2 x_3
 \end{aligned} \quad (28)$$

Conjunctive Normal Form. The bitwise AND operations are not supported by SAT solvers. As in [27], we employ some intermediate variables to represent those monomials such as $x_0 x_2 x_3$ that have AND operations. We denote the intermediate variable as I_p .

$$I_p = x_0^{t_p^0} x_1^{t_p^1} \dots x_{w-1}^{t_p^{w-1}}, \quad (p = 0, \dots, 2^w - 1) \quad (29)$$

Then we need to transform Eq. (29) into CNF. Consider the equation below:

$$\begin{aligned}
 (x_0 \vee \neg I_p) \wedge (x_1 \vee \neg I_p) \wedge \dots \wedge (x_{w-1} \vee \neg I_p) \wedge \\
 (\neg x_0 \vee \neg x_1 \vee \dots \vee \neg x_{w-1} \vee I_p) = 1
 \end{aligned} \quad (30)$$

Under the constraint of Eq. (30), for every different value of x_i , the value of I_p will be equal to the term $x_0^{t_p^0} x_1^{t_p^1} \dots x_{w-1}^{t_p^{w-1}}$. Eq. (30) can be supported by the solver. Therefore, by using $2^w - 1$ intermediate variables and adding $2^w - 1$ clauses into the solver, we can transform the ANF of the S-box to CNF.

For PRESENT, $w = 4$, if we map the terms in Eq. (27) to the intermediate variables I_p like in Table 2, we will have Eq. (31), which is in CNF.

$$\begin{aligned}
 y_0 &= I_1 + I_3 + I_4 + I_8 \\
 y_1 &= I_2 + I_4 + I_9 + I_{10} + I_{11} + I_{12} + I_{13} \\
 y_2 &= I_0 + I_3 + I_4 + I_5 + I_7 + I_9 + I_{12} + I_{13} \\
 y_3 &= I_0 + I_1 + I_2 + I_4 + I_8 + I_{11} + I_{12} + I_{13}
 \end{aligned} \quad (31)$$

4.2.3. Representation of permutation

The input and output of the permutation layer can be one bit to one bit or multiple bits to one bit. For the first case, the permutation layer usually uses the permutation table T_p to represent the relationship between input and output:

$$x_i + y_{T_p[i]} = 0, \quad 0 \leq i < n \quad (32)$$

The second case is related to multiplication on finite fields. It is essentially a truth table. So the method in Section 4.2.2 can be used.

Table 3

All cases for PRESENT-128.

Case	Average number of enumerations	Average number of ciphertexts
1	0	15
2	4.00	24.06
3	32.00	24.06
4	2.11	24.06

4.3. Algebraic representation of double persistent faults

For the S-box with double faults, firstly we use the method in Section 3.3 to get its truth table. Secondly, we represent it with ANF. And thirdly we transform the ANF to CNF as shown in Section 4.2.2. In algebraic analysis, we can exploit the fault leakage for multiple rounds by adding the constraints:

$$\tilde{K}_j^r \neq \tilde{Y}_j^r \oplus V, 0 \leq j < w \quad (33)$$

r ranges from R back to the round we use.

5. Experiment

PRESENT cryptographic algorithm has a relatively simple hardware implementation, so it is called a super lightweight cryptographic algorithm. AES is one of the most widely used and extensively analyzed block ciphers. We apply our attack model to PRESENT-128 and AES-128. Our results show that the number of ciphertexts we need to recover the key is reduced by 46% compared to APFA under a single fault.

The experiment of the fault attack is on a PC with 16 GB memory and an Intel i7-12700H CPU at 2.3 GHz. The operating system is a 64-bit Windows 11. Both PRESENT-128 and AES-128 are software implementations with Python. In the fault injection step, we simulated the attacker by randomly choosing two elements in the S-box and changing the value of the two elements.

5.1. Experimental results for PRESENT-128

We evaluated the average number of ciphertexts required for MLE under double faults. We performed 5000 simulation experiments for random double faults, and the average number of ciphertexts we need to recover the key is about 24.06. Table 3 shows the results of four cases.

In Cases 2, 3 and 4, we all use MLE to recover the faults. As in Eq. (20), MLE is only related to C_j^{min} and C_j^{max} , so the average number of faulty ciphertexts used is the same. In case 1 with known fault locations and fault values, We can directly build the equations and solve them. Compared to APFA under a single fault, we reduce 13 ciphertexts (APFA requires 28 ciphertexts), which is 46.43%. In Cases 2 and 3, as we have shown above, 4 and 32 enumerations are required and only one is solvable, i.e., the actual double faults. In Case 4, we introduce the method of chosen plaintexts, its average number of enumerations is about 2.11. In addition to the results of double faults in different cases, we also compare it with a single fault, which is shown in Fig. 5.

Fig. 5 shows the solving time for different faults with a fixed number of ciphertexts. The red curve is the solving time for a single fault and the black line is the result for double faults. When the fault leakage is sufficient (34 extra ciphertexts), the solving time is the same for both. Since the fault leakages of double faults are twice that of a single fault, double faults can still have enough fault leakages to recover the key when there are not enough ciphertexts. Therefore, in Case 1, the number of ciphertexts required for double faults is reduced by nearly half compared to a single fault.

Table 4

All cases for AES-128.

Case	Average number of enumerations	Average number of ciphertexts
1	0	700
2	4.00	982.41
3	512.00	982.41
4	2.23	982.41

Table 5

Efficiency comparison of AES with related work.

Analysis method	Number of ciphertexts	Percentage reduction
PFA-18	2281	–
PFA-20	1641	28.06%
EPFA	1000	56.16%
APFA(Single fault)	1300	43.01%
APFA(Double faults)	700	69.31%

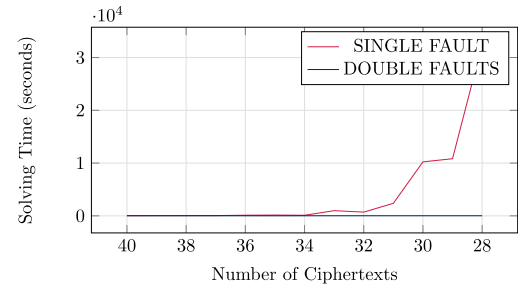


Fig. 5. Solving time and number of ciphertexts for different faults.

5.2. Experimental results for AES-128

In addition to lightweight block ciphers, AES-128 has also been analyzed and experimented. We evaluated the average number of ciphertexts required for each of the four cases. We conducted 5000 simulation experiments and calculated the average number of the required ciphertexts. Table 4 is the result.

In case 1 with known fault locations and fault values, we need 700 ciphertexts to recover the key. In Cases 2 and 3, 4 and 512 enumerations are required. In Case 4, by introducing the method of chosen plaintexts (Algorithm 1), its average number of enumerations is also about 2.11.

Table 5 shows the comparison between several related works and the original PFA [10]. PFA-20 [18] introduced MLE, which can reduce the number of ciphertexts by 28.06%. And Enhanced Persistent Fault Analysis (EPFA) [30] used the penultimate round of fault leakages to reduce the ciphertext by 56.16%. The first three methods are all manual analysis. APFA combines algebraic analysis and PFA, but due to the limitation of algebraic analysis itself, APFA can only use the last round of fault leakage. The effect of APFA under a single fault is not as good as EPFA, but under double faults, APFA reduces the ciphertexts by about 70% compared to the original PFA. Compared to APFA under a single fault, which needs 1300 ciphertexts, the double faults model only needs 700 ciphertexts, which reduces the number by 46.15%.

6. Conclusion

We proposed the double laser-faults based persistent fault attack on the cryptographic circuit. We exploit algebraic analysis to recover the key of block ciphers with double faults injection. As the nodes on the integrated circuit get smaller, some low-cost fault injection methods often induce more than one fault per injection. The proposed method can deal with the double faults model and reduce the number of required ciphertexts. The study extends the fault model in [22]. We experimented with PRESENT-128 and AES-128. The ciphertexts needed for recovering the key are reduced by 46% compared to an algebraic persistent fault attack with a single fault.

CRedit authorship contribution statement

Xue Gong: Conceptualization, Methodology, Formal analysis, Software. **Ao Shen:** Methodology, Software. **Tianxiang Feng:** Data curation, Software, Writing – original draft. **Guorui Xu:** Data curation, Validation. **Shize Guo:** Validation, Writing – review & editing. **Fan Zhang:** Conceptualization, Methodology, Validation, Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Fan Zhang reports administrative support was provided by Zhengzhou Xinda Institute of Advanced Technology.

Data availability

No data was used for the research described in the article.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (62227805, 62072398), by the Natural Science Foundation of Jiangsu Province (BK20220075) and by the Fok Ying-Tung Education Foundation for Young Teachers in the Higher Education Institutions of China (No. 20193218210004).

References

- [1] P.C. Kocher, *Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems*, Springer-Verlag, 1999.
- [2] X. jie Zhao, S. Guo, F. Zhang, T. Wang, Z.J. Shi, H. Liu, K. Ji, J. Huang, Efficient Hamming weight-based side-channel cube attacks on PRESENT, *J. Syst. Softw.* 86 (2013) 728–743.
- [3] D. Boneh, R.A. DeMillo, R.J. Lipton, On the importance of checking cryptographic protocols for faults, in: *International Conference on the Theory and Applications of Cryptographic Techniques*, Springer, 1997, pp. 37–51.
- [4] P. Dusart, G. Letourneux, O. Vivolo, Differential fault analysis on AES, in: *International Conference on Applied Cryptography and Network Security*, Springer, 2003, pp. 293–306.
- [5] M. Joye, M. Tunstall, *Fault Analysis in Cryptography*, vol. 147, Springer, 2012.
- [6] X. Li, R. Tessier, D. Holcomb, Precise fault injection to enable DFIA for attacking AES in remote FPGAs, in: *2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines, FCCM, 2022*, pp. 1–5, <http://dx.doi.org/10.1109/FCCM53951.2022.9786154>.
- [7] C. Aumüller, P. Bier, W. Fischer, P. Hofreiter, J.-P. Seifert, Fault attacks on RSA with CRT: Concrete results and practical countermeasures, in: *International Workshop on Cryptographic Hardware and Embedded Systems*, Springer, 2002, pp. 260–275.
- [8] S.P. Skorobogatov, R.J. Anderson, Optical fault induction attacks, in: *International Workshop on Cryptographic Hardware and Embedded Systems*, Springer, 2002, pp. 2–12.
- [9] S. Skorobogatov, Optical fault masking attacks, in: *2010 Workshop on Fault Diagnosis and Tolerance in Cryptography, IEEE, 2010*, pp. 23–29.
- [10] F. Zhang, X. Lou, X. Zhao, S. Bhasin, W. He, R. Ding, S. Qureshi, K. Ren, Persistent fault analysis on block ciphers, *IACR Trans. Cryptogr. Hardw. Embed. Syst.* (2018) 150–172.
- [11] E. Biham, A. Shamir, Differential cryptanalysis of DES-like cryptosystems, *J. Cryptology* 4 (1) (1991) 3–72.
- [12] E. Biham, A. Shamir, Differential fault analysis of secret key cryptosystems, 1997.
- [13] D. Mukhopadhyay, An improved fault based attack of the advanced encryption standard, in: *International Conference on Cryptology in Africa*, Springer, 2009, pp. 421–434.
- [14] G. Piret, J.-J. Quisquater, A differential fault attack technique against SPN structures, with application to the AES and KHAZAD, in: *International Workshop on Cryptographic Hardware and Embedded Systems*, Springer, 2003, pp. 77–88.
- [15] M. Tunstall, D. Mukhopadhyay, S. Ali, Differential fault analysis of the advanced encryption standard using a single fault, in: *IFIP International Workshop on Information Security Theory and Practices*, Springer, 2011, pp. 224–233.
- [16] A. Bogdanov, L.R. Knudsen, G. Leander, C. Paar, A. Poschmann, M.J. Robshaw, Y. Seurin, C. Viskelsøe, PRESENT: An ultra-lightweight block cipher, in: *International Workshop on Cryptographic Hardware and Embedded Systems*, Springer, 2007, pp. 450–466.
- [17] Z. Xinjie, W. Tao, G. Shize, Further improved deep differential fault analysis on camellia, in: *2012 Second International Conference on Instrumentation, Measurement, Computer, Communication and Control*, 2012, pp. 878–882, <http://dx.doi.org/10.1109/IMCCC.2012.211>.
- [18] F. Zhang, Y. Zhang, H. Jiang, X. Zhu, S. Bhasin, X. Zhao, Z. Liu, D. Gu, K. Ren, Persistent fault attack in practice, *IACR Trans. Cryptogr. Hardw. Embed. Syst.* (2020) 172–195.
- [19] T. Fuhr, É. Jaulmes, V. Lomné, A. Thillard, Fault attacks on AES with faulty ciphertexts only, in: *2013 Workshop on Fault Diagnosis and Tolerance in Cryptography, IEEE, 2013*, pp. 108–118.
- [20] C. Dobraunig, M. Eichlseder, T. Korak, S. Mangard, F. Mendel, R. Primas, SIFA: exploiting ineffective fault inductions on symmetric cryptography, *IACR Trans. Cryptogr. Hardw. Embed. Syst.* (2018) 547–572.
- [21] N.T. Courtois, K. Jackson, D. Ware, Fault-algebraic attacks on inner rounds of DES, in: *E-Smart'10 Proceedings: The Future of Digital Security Technologies, Strategies Telecom and Multimedia*, 2010.
- [22] F. Zhang, T. Feng, Z. Li, K. Ren, X. Zhao, Free fault leakages for deep exploitation: Algebraic persistent fault analysis on lightweight block ciphers, *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2022 (2) (2022) 289–311, URL: <https://tches.iacr.org/index.php/TCHES/article/view/9489>.
- [23] S. Engels, F. Schellenberg, C. Paar, SPFA: SFA on multiple persistent faults, in: *2020 Workshop on Fault Detection and Tolerance in Cryptography, FDTC, IEEE, 2020*, pp. 49–56.
- [24] H. Soleimany, N. Bagheri, H. Hadipour, P. Ravi, S. Bhasin, S. Mansouri, Practical multiple persistent faults analysis, *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2022 (1) (2021) 367–390.
- [25] M. Bardet, J. Faugère, B. Salvy, On the complexity of the F5 Gröbner basis algorithm, *J. Symbolic Comput.* 70 (2015) 49–70, <http://dx.doi.org/10.1016/j.jsc.2014.09.025>.
- [26] N.T. Courtois, Algebraic attacks on combinators with memory and several outputs, in: *International Conference on Information Security and Cryptology*, Springer, 2004, pp. 3–20.
- [27] M. Soos, K. Nohl, C. Castelluccia, Extending SAT solvers to cryptographic problems, in: *International Conference on Theory and Applications of Satisfiability Testing*, Springer, 2009, pp. 244–257.
- [28] N. Een, Minisat : A sat solver with conflict-clause minimization, in: *Proc Sat-05: International Conference on Theory & Applications of Satisfiability Testing*, 2005.
- [29] L.R. Knudsen, C.V. Miolane, Counting equations in algebraic attacks on block ciphers, *Int. J. Inf. Secur.* 9 (2) (2010) 127–135.
- [30] G. Xu, F. Zhang, B. Yang, X. Zhao, W. He, K. Ren, Pushing the limit of PFA: Enhanced persistent fault analysis on block ciphers, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* (2020).